

IBM University Engagement Project Submission

“Exploring the Power of Agentic AI with IBM Granite and LangFlow”

Project Title - Predictive Maintenance of Industrial Machinery using ML
Domain of Project – Manufacturing/ Heavy Industry

Student Name	Recent Passport Photo	Email ID	Mobile number [WhatsApp]
Tejaveni Chillapalli		chtejaveni@gmail.com	8143210885

Problem Statement -

Problem Statement : Predictive Maintenance of Industrial Machinery

The Challenge

- Unexpected equipment breakdowns cause costly unscheduled downtime and production delays.
- Traditional maintenance relies on fixed schedules (preventive) or waiting for failure (reactive), which is inefficient and expensive.
- Analyzing complex industrial sensor data (temperature, voltage, vibration) in real time is difficult without automated patterns.

The Objective

Build a machine learning-powered predictive maintenance pipeline that analyzes industrial machinery data to anticipate failures before they occur. The solution aims to:

- **Predict System Failures:** Classify equipment status in real time to distinguish between normal operating conditions and impending failures.
- **Identify Failure Root Causes:** Analyze critical sensor metrics (like temperature, torque, and speed) to isolate exactly why a machine component might break down.
- **Reduce Downtime & Costs:** Shift maintenance workflows from reactive/preventive to proactive, minimizing operational disruptions and saving on sudden replacement costs.
- **Provide Actionable Insights:** Give maintenance teams clear, data-backed early warnings to schedule repairs precisely when needed.

Technology Used

1. Programming Language & Core Frameworks

Python: The core language used for data engineering, model development, and scripting the user interface.

Streamlit: An open-source framework used to build the interactive web-based dashboard that acts as the user-facing application for the predictive maintenance system.

2. Cloud Infrastructure & Model Deployment

IBM Cloud (Watson Machine Learning / Cloud Object Storage): * Serves as the robust cloud platform where the trained predictive model is hosted and deployed as a live REST API endpoint.

Manages the secure computing infrastructure required to handle inference requests at scale.

API Integration: The Streamlit application connects to the **IBM Cloud Deployment URL**, securely passing real-time sensor metrics to the cloud and fetching immediate breakdown predictions.

3. Machine Learning & Data Science Stack

Scikit-Learn: Core library used for data preprocessing, train-test splitting, feature scaling, and training classification algorithms.

Pandas & NumPy: Used for intensive data manipulation, handling tabular sensor arrays, and restructuring telemetry inputs before sending them to the IBM Cloud API.

4. Local Prototyping & Tools

VS Code (Visual Studio Code): The primary Integrated Development Environment (IDE) used for writing clean code, modularizing scripts, and managing local testing.

Git & GitHub: Utilized for codebase version control and tracking changes throughout the development cycle.

Proposed Solution -

Proposed Solution: Predictive Maintenance of Industrial Machinery using ML

Overview

The proposed solution is an **End-to-End Machine Learning Pipeline** designed to monitor industrial machinery health by analyzing real-time sensor data. By transforming raw telemetry (temperature, speed, torque, voltage) into actionable insights, the system moves industrial operations from expensive reactive maintenance to data-driven proactive decision-making.

Core Components of the Solution

- **Data Engineering & Preprocessing:**

Ingests multi-sensor time-series or tabular data streams.

Cleans, scales, and transforms raw sensor inputs to eliminate noise and handle missing data.

- **Feature Engineering & Selection:**

Extracts critical statistical indicators and identifies correlations between sensor thresholds (e.g., rapid spikes in temperature or sudden drops in voltage).

- **Predictive ML Modeling:**

Leverages optimized classification algorithms to evaluate machine status in real time.

Predicts the likelihood of an imminent system failure and categorizes the specific type of fault occurring.

- **Evaluation & Optimization:**

Validates model precision and recall metrics to ensure high accuracy while drastically minimizing costly false alarms for engineering teams.

- **Key Benefits**

- **Minimized Downtime:** Fixes issues *before* a catastrophic failure forces a production line to stop.

- **Extended Equipment Lifespan:** Prevents secondary damage to surrounding machine components.

- **Optimized Workflows:** Schedules maintenance windows during planned, off-peak operational hours.

Algorithms and Code Snippets -

The screenshot displays a Jupyter Notebook interface in Google Colab, titled "Machine Learning project for AI and IISB cloud". The notebook is organized into several cells, with the first cell containing a list of tasks:

1. filter out 'Random failures' because they are statistically unpredictable
2. Re-split features and target
3. Inverse target names cleanly
4. Re-split test split
5. Re-train Random Forest WITHOUT SMOTE, using Internal class balancing Instead
6. Predict and print the results, uncalibrated report
7. Print the results, calibrated report
8. Print the results, calibrated report

The second cell contains the following Python code:

```
df_filtered = df_clean[df_clean['failure_type'] != 'Random failures'].copy()

X_filtered = df_filtered.drop(columns=['target', 'failure_type'])
y_filtered = df_filtered['target']

X_train, X_test, y_train, y_test = train_test_split(
    X_filtered, y_filtered, test_size=0.2, random_state=42, stratify=y_filtered)

rf_balanced = RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42)
rf_balanced.fit(X_train, y_train)

y_pred = rf_balanced.predict(X_test)

print("===== New Cleaned Classification Report =====")
print(classification_report(y_test, y_pred, target_names=class_names_filtered))
```

The third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The tenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eleventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twelfth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fourteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventeenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The nineteenth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twentieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The twenty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirtieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The thirty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fortieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The forty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fiftieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The fifty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixtieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The sixty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The seventy-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eightieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-first cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-second cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-third cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-fourth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-fifth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-sixth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-seventh cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-eighth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The eighty-ninth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The ninetieth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The hundredth cell contains the following Python code:

```
print("===== New Cleaned Classification Report =====")
```

The notebook also includes a table showing the results of the classification report:

	precision	recall	f1-score	support
Heat Dissipation failure	0.92	1.00	0.96	22
No failure	1.00	1.00	1.00	1933
Overvoltage failure	0.94	1.00	0.97	16
Power failure	1.00	1.00	1.00	19
Tool heat failure	0.80	0.80	0.80	9
accuracy			0.99	1955
macro avg	0.77	0.88	0.78	1957
weighted avg	0.95	0.90	0.96	1957

The notebook also includes a section for the "New Cleaned Classification Report" with the following results:

Heat Dissipation failure: 0.92 precision, 1.00 recall, 0.96 f1-score, 22 support

No failure: 1.00 precision, 1.00 recall, 1.00 f1-score, 1933 support

Overvoltage failure: 0.94 precision, 1.00 recall, 0.97 f1-score, 16 support

Power failure: 1.00 precision, 1.00 recall, 1.00 f1-score, 19 support

Tool heat failure: 0.80 precision, 0.80 recall, 0.80 f1-score, 9 support

accuracy: 0.99

macro avg: 0.77 precision, 0.88 recall, 0.78 f1-score

weighted avg: 0.95 precision, 0.90 recall, 0.96 f1-score

The notebook also includes a section for the "New Cleaned Classification Report" with the following results:

Heat Dissipation failure: 0.92 precision, 1.00 recall, 0.96 f1-score, 22 support

No failure: 1.00 precision, 1.00 recall, 1.00 f1-score, 1933 support

Overvoltage failure: 0.94 precision, 1.00 recall, 0.97 f1-score, 16 support

Power failure: 1.00 precision, 1.00 recall, 1.00 f1-score, 19 support

Tool heat failure: 0.80 precision, 0.80 recall, 0.80 f1-score, 9 support

accuracy: 0.99

macro avg: 0.77 precision, 0.88 recall, 0.78 f1-score

weighted avg: 0.95 precision, 0.90 recall, 0.96 f1-score

[illegible]

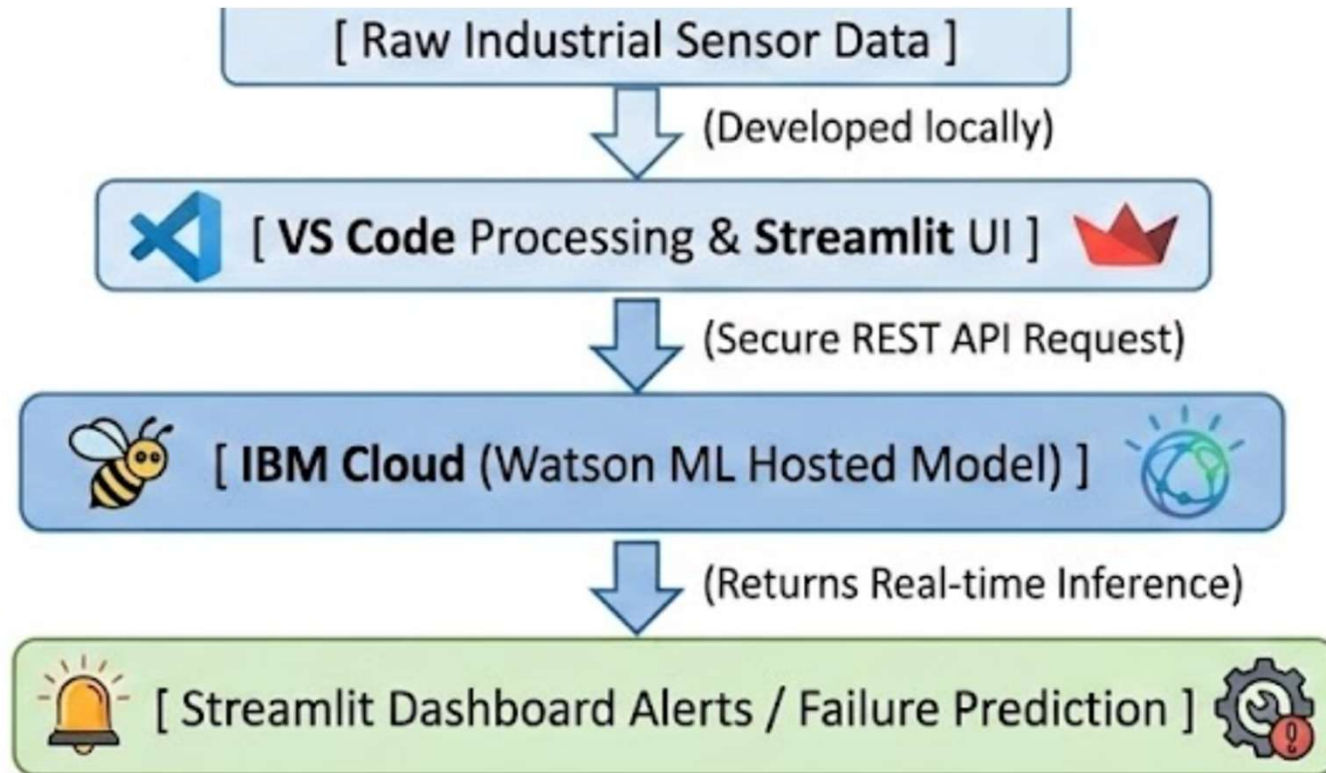
The screenshot displays a Jupyter Notebook environment with the following components:

- File Explorer:** Shows the notebook's file structure, including 'Predictive_maintenance.ipynb' and various data files.
- Code Cell:** Contains Python code for:
 - Importing necessary libraries (sklearn, numpy, pandas, etc.).
 - Loading and preprocessing the 'Predictive_maintenance.ipynb' dataset.
 - Splitting the data into training and testing sets.
 - Training a Random Forest Classifier with a custom loss function ('loss_weighted_balanced_subsample') that penalizes misclassification of 'Failure'.
 - Evaluating the model's performance using a custom metric ('y_custom_s') and printing the results.
- Output:** The final output is a table summarizing the model's performance:

	precision	recall	f1-score	support
Hot Machine Failures	0.01	0.00	0.01	11

A notification at the bottom right indicates that the extension host was terminated unexpectedly 3 times within the last 10 minutes, with a button to 'Restart Extension Host'.

Architecture blueprint



Role of Smart Automation and Cloud Intelligence in Predictive Maintenance

- **Autonomous Machine Health Monitoring:** Instead of waiting for a human operator to look at raw data, the deployed IBM Cloud model acts as an autonomous digital inspector. It continuously evaluates incoming sensor metrics and flags risks without manual intervention.
- **Context-Aware Inference:** The predictive system understands the complex, combined context of multiple telemetry factors (such as matching a high temperature reading with a sudden drop in voltage) to intelligently deduce the specific state of the machinery.
- **Proactive Rather Than Reactive:** True to agentic principles, the solution transitions operations from a static, defensive posture to a proactive, goal-driven workflow. It provides early warnings to maintenance teams before a catastrophic breakdown can occur.
- **Decoupled Architecture Integration:** By hosting the model as a live REST API endpoint on IBM Cloud, the analytical engine functions independently from the Streamlit interface. This creates a scalable framework where the "AI Agent" can easily serve multiple client interfaces or factory floors simultaneously

Novelty and Uniqueness

1. Cloud-Native & Production-Ready Architecture

Beyond Local Execution: Unlike standard student projects that run entirely on a local machine, this solution is decoupled. The core intelligence is fully containerized and hosted in the cloud.

Seamless API-Driven Inference: The Streamlit interface acts as a lightweight client that communicates with a live **IBM Cloud REST API endpoint**, showcasing an enterprise-ready, scalable architecture.

2. End-to-End MLOps Pipeline Implementation

Separation of Concerns: Development and deployment are treated as distinct phases. Code is modularly authored in **VS Code**, version-controlled on **GitHub**, and systematically deployed to **IBM Cloud**.

Robust Lifecycle Management: By utilizing IBM's cloud infrastructure, the system lays the groundwork for seamless model versioning, monitoring, and regular payload logging without breaking the user-facing application.

3. Real-Time, Multi-Sensor Telemetry Analysis

Holistic Machinery Health Profiling: The model doesn't just look at one metric in isolation; it dynamically evaluates complex correlations across multiple parallel sensor inputs (e.g., temperature spikes combined with sudden voltage drops).

Early-Warning Classification: Instead of relying on a simple binary warning (Working vs. Broken), the solution is designed to classify impending faults early, giving maintenance teams a critical window to take proactive action.

4. Accessible & Intuitive Interface for Field Engineers

Data Democratization: By wrapping a powerful machine learning model in a clean, interactive **Streamlit dashboard**, complex predictive analytics are translated into simple, visual health alerts that factory floor workers can instantly interpret.

Github Link and Other Links

Github Link - <https://github.com/Teju-2007/Predictive-Maintenance-ML.git>

Streamlit App Link - <https://predictive-maintenance-ml-hminumxwwprhfgueue9vdyd.streamlit.app/>

.

.

Future Scope

1. Real-Time IoT Sensor Integration

Edge Computing Deployments: Expand the architecture to ingest live, continuous telemetry streams directly from physical IoT sensors (such as vibration, temperature, and acoustic sensors) embedded in active industrial machinery.

Continuous Streaming Pipelines: Transition the system from a batch-input Streamlit interface to a real-time event-driven data streaming model using Apache Kafka or IBM Event Streams.

2. Advanced Multi-Modal Deep Learning

Acoustic & Thermal Analysis: Incorporate audio data (for detecting unusual mechanical grinding) and thermal imaging maps into the predictive models alongside traditional numerical sensor metrics.

Anomaly Detection using Autoencoders: Implement unsupervised deep learning models to flag unknown or unprecedented machine behaviors that standard classification models might miss.

3. Explainable AI (XAI) & Root-Cause Diagnosis

IBM Watson OpenScale Integration: Use Explainable AI frameworks (like SHAP or LIME) via IBM Cloud to not only predict *when* a machine will fail but explicitly explain *which* specific sensor threshold caused the warning.

Automated Prescription Actions: Upgrade the system to generate automatic maintenance work orders, detailing exactly what parts need to be repaired or replaced upon fault detection.

4. Cross-Fleet Scaling & Mobile Alerting

Fleet-Wide Fleet Monitoring Dashboard: Scale the interface to handle multiple industrial plants, letting managers view the real-time health index of thousands of machines from a centralized map.

Instant Alert System: Integrate automated notification systems (SMS, email, or Slack/Microsoft Teams webhooks) to ping maintenance technicians the moment the IBM Cloud endpoint flags an imminent critical failure.

.

Thank You!

Thank you for your time and interest.